

Parallelization Approach and Performance Analysis

Group 30

ist103811 - Tiago Deane
ist102703 - Isabela Pereira
ist99044 - Adrian Graur

April 2025

0 Introduction

This report will dive into the parallelization approach we used in our project. We will discuss aspects such as our concerns about synchronization, the decomposition used, load balancing techniques, and finally our conclusions about the performance obtained after the parallelization and the issues and potential limitations in our implementation. We include a tabular representation with our performance results where we show the execution times obtained from our serial implementation as well as with for 1, 2, 4, 8, 16, 32 and 64 parallel tasks and two graphs, one visually representing the speedup obtained and the other on the time of execution. We will also dive into the corrections we made from the last project delivery concerning the OpenMP implementation.

0.5 Corrections to OpenMP

Since the last project delivery, we were able to fix some of the errors we had. We started by removing all the instances where we used `pragma omp critical` and now we use a lock mechanism. Also, for cases that were too big, we were getting a segmentation fault. We found that this happened because of the way we allocate memory in the grid. Now we use `malloc` and check if memory is being allocated before continuing. Furthermore, we were incorrectly calculating the center of mass before detecting collisions, so this was fixed in this version (though this increased execution times). Below we will show the new performance results compared to the last delivery.

Performance Results

The computer used for the original results was Lab2P8, with 2 thread(s) per core and 6 core(s) per socket. The computer used in the second set of results is Lab2P2, with 2 thread(s) per core and 6 core(s) per socket.

$T_{serial_old} = 156.2s$ $T_{serial_new} = 152.5s$				
Nº of Threads	T_{OMP_OLD}	Speedup 1	T_{OMP_NEW}	Speedup 2
1	159.2s	0.98	174.8s	0.87
2	86.5s	1.81	89.2s	1.71
4	53.3s	2.93	91.2s	1.67
6	42.2s	3.70	60.5s	2.52
8	43.8s	3.57	48.3s	3.15

Table 1: Results for the example `./parsim 12 100 5 10000 10000`

$T_{serial_old} = 244.7s$ $T_{serial_new} = 255.8s$				
Nº of Threads	T_{OMP_OLD}	Speedup 1	T_{OMP_NEW}	Speedup 2
1	245.6s	0.99	272.9s	0.94
2	125.2s	1.95	111.6s	2.29
4	63.7s	3.84	55.8s	4.58
6	44.6s	5.49	55.1s	4.64
8	44.9s	5.45	46.1s	5.55

Table 2: Results for the example `./parsim -11 3500 20 500000 10`

Performance Plots

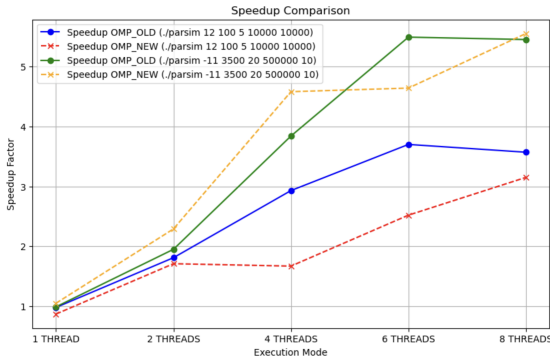


Figure 1: Speedup of Parallel Implementation.

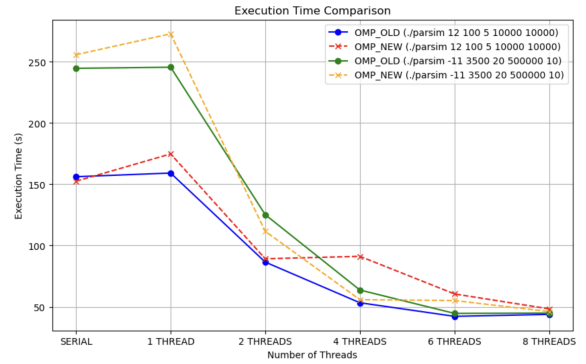


Figure 2: Time Comparison of Parallel Implementation.

1 Approach to Parallelization

The parallelization approach used in our code is a MPI strategy with domain decomposition, using grid row partitioning:

- Processes exchange center-of-mass with **above** and **below** ranks to compute the forces correctly near the boundaries;
- Particles moving out of a processes' row are sent to the other process using MPI message passing (`MPI_Isend`, `MPI_Irecv`);
- Process containing the main particle is tracked;
- For the collision detection, each process handles collisions locally, with a final reduction that aggregating the total number of collisions at the end (`MPI_Reduce`).

2 Decomposition Used

We decompose the domain into cells and agglomerate cells into rows, ensuring that each MPI Rank handles a subset of the rows. In this way, processes only need to communicate with two processes during the main part of the simulation, the one directly above and the one directly below their set of rows. For calculating the center of mass, each process sends its first row to the process handling the rows above and its last row to the process handling the rows below. This ensures that every process has the necessary information about its neighboring cells.

3 Synchronization Concerns

Each process operates within its assigned range of the grid, minimizing major synchronization issues. However, two key concerns were addressed:

- **Total Collision Reduction:** Each process maintains a local count of collisions occurring within its domain. At the end of the computation, all processes must aggregate their individual collision counts to obtain the total number of collisions throughout the system. To ensure a safe and efficient reduction of these values, we used *MPI_Reduce*, enabling a collective and synchronized summation of collision data across all processes. We also ensure that the main particle's location is tracked so that the appropriate process performs the reduction and prints the final output correctly.
- **Main Particle Tracking:** Each particle has a field called "id", which is a *char* that is 1 if the particle is the main particle and 0 otherwise. Our program keeps track of the process that has the main particle, and after finishing all timesteps this process will provide this information to the remaining processes, so that *MPI_Reduce* can be performed.

- **Inter-Process Particle Exchange:** The number of particles migrating to neighboring cells managed by other processes is not predetermined. This requires an additional synchronization step to ensure efficient communication between processes, allowing for proper buffer allocation and message handling. To achieve this, each process must first communicate the size of the buffer it intends to send before transmitting the actual particle data. This step allows the receiving process to allocate the appropriate memory and prepare for data reception, thus preventing issues such as buffer overflows or unexpected message sizes.

4 Load Balancing

To achieve proper load balancing we ensured that grid cells were distributed as evenly as possible across processes. We did this by giving each process a subset of sequential rows, balancing the number of rows handled by each process while simultaneously minimizing communication between processes. While this approach is beneficial in most scenarios, it may also introduce overhead in others.

5 Possible Problems and Limitations

In cases where the number of particles is extremely large and the board has a very big number of cells per side compared to the size of the simulation, some particles may move across multiple cells within a single timestep. Our implementation does not currently detect these cases, which may lead to incorrect results in very specific scenarios. We also noticed that we were calculating the cells' centers of mass after updating the particles but before detecting collisions, so when particles were removed as a result of collisions, this would not be reflected in the centers of mass at the time of the next *update_particles* call. We updated this in the Serial and OMP versions, as they fixed some results, but since we discovered this close to the deadline and they didn't fix our problems in the MPI version, we didn't update it.

6 Performance Results

The computer used for the serial result was Lab2P2, with 2 thread(s) per core and 6 core(s) per socket. For the MPI results, we utilized the RNL cluster with Slurm commands. The second table only tested up to 16 processes because there are 30 cells per side, and we did not support the case where there are more processes than cells per side.

$T_{serial} = 96.3s$		
Nº of Processes	T_{MPI}	Speedup
1	54.2s	1.77
2	34.1s	2.84
4	20.2s	4.77
8	9.2s	10.5
16	10.0s	9.63
32	4.5s	21.4
64	2.6s	37.04

Table 3: Results for the example `"/parsim 1 5000 100 1000000 100"`

$T_{serial} = 335.3s$		
Nº of Processes	T_{MPI}	Speedup
1	163.5s	1.53
2	85.5s	2.93
4	157.2s	1.59
8	126.0s	1.98
16	68.7s	3.64

Table 4: Results for the example `"/parsim -1 1000 30 100000 1000"`

Conclusions

We were able to observe very large speedups with our MPI implementation, usually increasing with the number of processes. In some cases, for specific numbers of processes the execution time increased, but we can attribute this to communication overhead between processes, or simply bad luck in terms of the particle distribution or movement of particles between rows, leading to increased communication overhead.

7 Performance Plots

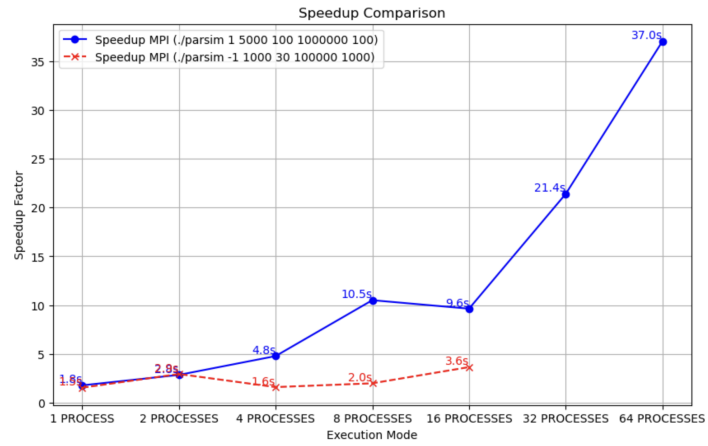


Figure 3: Speedup of Parallel Implementation.

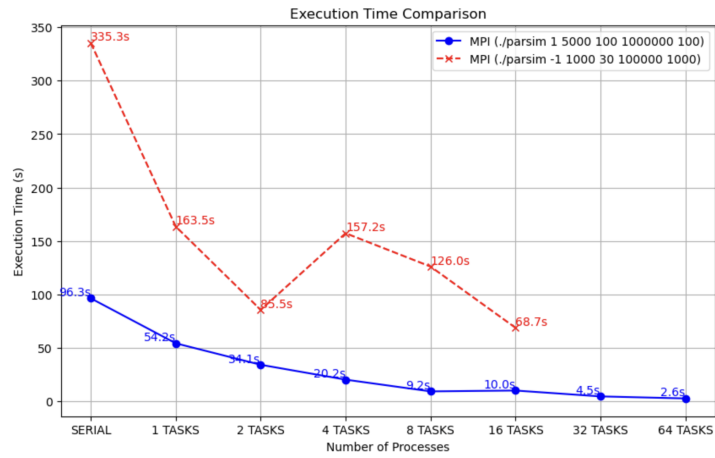


Figure 4: Time Comparison of Parallel Implementation.